

Subject: PRF193 - Programming Fundamentals (with C/C++)

Topic: Mini-Supermarket Management

I, Group member.

- Nguyen Ngoc Nam – DE200201
- Nguyen Hai Duong – DE200154
- Nguyen Do Tien Thinh – DE200232
- Le Nhat Thanh – De200496

II, Project Overview.

1. Objectives

The objective of this project is to design and implement a console-based Store Management System using C++. The system simulates basic operations of a small retail store, including product browsing, shopping cart handling, and transaction processing.

This project focuses on applying core programming concepts:

- Object-Oriented Programming (OOP):
Applying encapsulation, inheritance, and polymorphism through class design.
- Standard Template Library (STL):
Using vector to dynamically manage collections such as products, users, and cart items.
- Dynamic Memory Management:
Managing user objects using pointers and ensuring proper deallocation.
- System Simulation:
Implementing a multi-role system (Customer, Staff, Manager, Boss) to reflect real-world responsibilities.

2. System Overview

The system is designed as a **menu-driven console application** where users interact with the system based on their roles.

Main Components:

- Product Management
- User Authentication System
- Shopping Cart System
- Revenue Tracking System

User Roles and Permissions:

Role	Permissions
Customer	View products, search, add to cart, checkout
Staff	Perform quick selling (direct purchase without cart)
Manager	View product list only
Boss	View total system revenue

3. Functional Requirements

3.1 User Management

Attributes:

- Username (unique)
- Password
- Role

Functions:

- Registering new users (Customer or Staff)
- Login with username and password
- Role-based menu display after login

3.2 Product Management

- **Attributes:**

- Product ID
- Name
- Category
- Price
- Stock quantity

- **Functions:**

- Display all products
- Search product by name
- Update stock automatically after purchase

3.3 Cart & Purchase Management

Functions:

- Add products to cart
- View cart contents
- Calculate total price
- Checkout:
 - Reduce product stock
 - Update system revenue
 - Clear cart

3.4 Revenue Management

Functions:

- Track total revenue
- Update revenue after:
 - Customer checkout
 - Staff quick sell
- Boss can view total revenue

4. System Design

4.1 Class Structure

The system includes the following main classes:

- Product
- User (Abstract class)
- Customer, Staff, Manager, Boss (Derived classes)
- Store
- CartItem (struct)

4.2 Class Description

Product Class

Stores product information and provides methods to:

- Display product details
- Increase/decrease stock

User Class (Abstract)

- Contains common attributes: username, password, role
- Declares virtual function showMenu()

Derived Classes

- **Customer:**
Can browse products, search, add to cart, and checkout
- **Staff:**
Can perform quick selling by entering product ID and quantity
- **Manager:**
Only allowed to view product list
- **Boss:**
Can view total revenue of the system

Demonstrates **polymorphism** through overridden showMenu() method

Store Class

Acts as the **central controller** of the system.

Managers:

- List of products (vector<Product>)
- List of users (vector<User*>)
- Shopping cart (vector<CartItem>)
- Revenue tracking

5. Implementation Details

5.1 OOP Concepts

- **Encapsulation:**
Data is encapsulated within classes like Product and User
- **Inheritance:**
Derived classes inherit from base class User

- **Polymorphism:**
Different behaviors implemented via showMenu() in each role

5.2 STL Usage

- vector<Product> → manage product list dynamically
- vector<User*> → store multiple user types
- vector<CartItem> → manage cart items

5.3 Design Decisions

- **Using User* (pointer):**
Allows storing different user types in the same container and enables polymorphism.
- **Using CartItem:**
Stores a pointer to Product and quantity → avoids duplicating product data.
- **Using vector:**
Provides flexible resizing and easy iteration.

5.4 Memory Management

- Users are created dynamically using new
- Destructor in Store deletes all user objects → prevents memory leaks

5.5 Input/Output Handling

- Console-based interaction using cin and cout
- Output formatting using iomanip

6. Program Flow

6.1 Main Flow

Start → Main Menu (1.Login / 2.Register / 0.Exit)

→ If Register → Input username/password → Create user → Back to Main Menu

→ If Login → Input username/password → Validate account

→ If fail → Show error → Back to Main Menu

→ If success → Go to Role Menu

→ User selects function → Perform actions → Logout

→ Back to Main Menu → End

6.2 Customer Flow

View products / Search → Add to cart

→ View cart → Checkout → If cart empty → Show message → Back

→ Else → Check stock

→ If valid → Update stock & revenue → Clear cart

→ If invalid → Show error → Back to menu

6.3 Staff Flow

Enter product ID & quantity

→ Check product & stock

→ If valid → Reduce stock → Update revenue → Success

→ If invalid → Show error → Back to menu

7. Limitations & Future Improvements

Limitations

- No data persistence (data lost after program ends)
- No password encryption

Future Improvements

- Add file handling to save/load data
- Secure password system (encryption)

9. Conclusion

- This project implements a console-based store management system in C++ with multiple user roles such as Customer, Staff, Manager, and Boss. The system supports basic functions including product display, shopping cart, checkout, and revenue tracking.
- It demonstrates the application of object-oriented programming, STL containers, and dynamic memory management. The system can be further improved with features such as data storage and better user interface.